

ASSIGNMENT

Course:	Full Stack Web Development	Course Code:	23CM4121
Assignment No.:	1	Date:	30-08-2026
Student Name:	R Varshini	Roll No.:	A24126552115

1. TITLE

Implement Inheritance Types Using JavaScript

2. AIM

To implement and demonstrate the different types of inheritance in JavaScript — single, multilevel, hierarchical, multiple (via mixins), and hybrid inheritance — using ES6 class syntax.

3. OBJECTIVE

- To understand the concept of inheritance in JavaScript using ES6 classes.
- To implement single inheritance using the extends keyword.
- To implement multilevel inheritance by chaining classes across multiple generations.
- To implement hierarchical inheritance where multiple child classes extend a common parent class.
- To implement multiple inheritance in JavaScript using the mixin pattern, since classes cannot directly extend more than one class.
- To implement hybrid inheritance by combining classical inheritance (extends) with mixins.
- To verify the behaviour of each inheritance type by creating objects and invoking inherited methods.

4. THEORY

Inheritance is an object-oriented programming concept that allows one class (child/derived class) to acquire the properties and methods of another class (parent/base class). In JavaScript, ES6 introduced the class and extends keywords, which provide a cleaner, class-based syntax over the underlying prototype-based inheritance model.

JavaScript supports the following types of inheritance:

- **Single Inheritance** — A single child class inherits from a single parent class using extends. For example, class Dog extends Animal inherits the eat() method from Animal.
- **Multilevel Inheritance** — A class inherits from a class which itself inherits from another class, forming a chain (Animal2 → Dog2 → Puppy). Each level adds its own methods while retaining access to all methods above it.
- **Hierarchical Inheritance** — Multiple child classes (Dog3, Cat3) independently extend the same parent class (Animal3), each inheriting the parent's methods while adding their own distinct behaviour.

- Multiple Inheritance — JavaScript classes can extend only one class directly, so true multiple inheritance is achieved using the mixin pattern: a mixin is a function that takes a base class and returns a new class extending it with additional methods. Chaining mixins (e.g. `MotherMixin(FatherMixin(Object))`) lets a class combine behaviour from multiple sources.
- Hybrid Inheritance — A combination of more than one inheritance type in the same design — for example, classical inheritance (`Human` extends `LivingThing`) combined with mixins (`StudentMixin`, `WorkerMixin`) applied on top of `Human` to build `Student` and `Employee` classes.

In all cases, the child class gains access to the parent's (and, in the case of mixins, the mixed-in) methods without redefining them, which promotes code reuse and a clear object hierarchy.

5. ALGORITHM / PROCEDURE

- Create a JavaScript file to hold all five inheritance demonstrations.
- For single inheritance: define a parent class `Animal` with an `eat()` method, and a child class `Dog` that extends `Animal` and adds a `bark()` method. Create a `Dog` object and call both methods.
- For multilevel inheritance: define `Animal2` → `Dog2` → `Puppy` as a three-level chain, each adding one new method. Create a `Puppy` object and call all three inherited/own methods.
- For hierarchical inheritance: define a parent class `Animal3`, and two child classes `Dog3` and `Cat3` that both extend `Animal3` independently. Create objects of both children and call their methods.
- For multiple inheritance: define mixin functions `FatherMixin` and `MotherMixin` that each take a base class and return an extended class. Create a `Child` class that extends `MotherMixin(FatherMixin(Object))` and combines properties from both mixins with its own.
- For hybrid inheritance: define a base class `LivingThing` and a class `Human` that extends it. Define `StudentMixin` and `WorkerMixin`, then create `Student` (extends `StudentMixin(Human)`) and `Employee` (extends `WorkerMixin(Human)`) to combine classical inheritance with mixins.
- Instantiate objects for each demonstration and call their inherited and own methods.
- Run the file using Node.js and observe the console output for each inheritance type.
- Verify the actual output against the expected output for each section.

6. PROGRAM

ass.js

```
// =====
// 1. SINGLE INHERITANCE
// =====
class Animal {
  eat() {
    console.log("Animal eats");
  }
}
class Dog extends Animal {
  bark() {
    console.log("Dog barks");
  }
}
console.log("1. Single Inheritance:");
let dog = new Dog();
dog.eat();
dog.bark();
```

```

// =====
// 2. MULTILEVEL INHERITANCE
// =====
class Animal2 {
  eat() {
    console.log("Animal eats");
  }
}
class Dog2 extends Animal2 {
  bark() {
    console.log("Dog barks");
  }
}
class Puppy extends Dog2 {
  play() {
    console.log("Puppy plays");
  }
}
console.log("\n2. Multilevel Inheritance:");
let puppy = new Puppy();
puppy.eat();
puppy.bark();
puppy.play();

// =====
// 3. HIERARCHICAL INHERITANCE
// =====
class Animal3 {
  eat() {
    console.log("Animal eats");
  }
}
class Dog3 extends Animal3 {
  bark() {
    console.log("Dog barks");
  }
}
class Cat3 extends Animal3 {
  meow() {
    console.log("Cat meows");
  }
}
console.log("\n3. Hierarchical Inheritance:");
let dog3 = new Dog3();
dog3.eat();
dog3.bark();
let cat3 = new Cat3();
cat3.eat();
cat3.meow();

// =====
// 4. MULTIPLE INHERITANCE
// Using Mixins
// =====

```

```

class Father {
  fatherProperty() {
    console.log("Property from Father");
  }
}
class Mother {
  motherProperty() {
    console.log("Property from Mother");
  }
}
// Mixin functions
const FatherMixin = Base => class extends Base {
  fatherProperty() {
    console.log("Property from Father");
  }
};
const MotherMixin = Base => class extends Base {
  motherProperty() {
    console.log("Property from Mother");
  }
};
class Child extends MotherMixin(FatherMixin(Object)) {
  childProperty() {
    console.log("Property from Child");
  }
}
console.log("\n4. Multiple Inheritance:");
let child = new Child();
child.fatherProperty();
child.motherProperty();
child.childProperty();

// =====
// 5. HYBRID INHERITANCE
// Combination of inheritance types
// =====
class LivingThing {
  live() {
    console.log("Living thing lives");
  }
}
class Human extends LivingThing {
  walk() {
    console.log("Human walks");
  }
}
const StudentMixin = Base => class extends Base {
  study() {
    console.log("Student studies");
  }
};
const WorkerMixin = Base => class extends Base {
  work() {
    console.log("Worker works");
  }
}

```

```

};
class Student extends StudentMixin(Human) {
  write() {
    console.log("Student writes");
  }
}
class Employee extends WorkerMixin(Human) {
  workAtOffice() {
    console.log("Employee works at office");
  }
}
console.log("\n5. Hybrid Inheritance:");
let student = new Student();
student.live();
student.walk();
student.study();
student.write();
let employee = new Employee();
employee.live();
employee.walk();
employee.work();
employee.workAtOffice();

```

7. CODE EXPLANATION

Code	Explanation
<code>class ... extends ...</code>	Creates a child class that inherits all properties and methods of the specified parent class.
<code>class Dog extends Animal</code>	Single inheritance: Dog directly inherits <code>eat()</code> from <code>Animal</code> and adds <code>bark()</code> .
<code>Animal2 → Dog2 → Puppy</code>	Multilevel inheritance: Puppy inherits from Dog2, which itself inherits from Animal2, forming a chain.
<code>Dog3, Cat3 extends Animal3</code>	Hierarchical inheritance: two independent child classes inherit from the same parent class.
<code>Base => class extends Base {...}</code>	A mixin function that dynamically creates a new class extending whatever base class is passed to it.
<code>MotherMixin(FatherMixin(Object))</code>	Chains two mixins onto Object to simulate multiple inheritance, since a class can only extend one class directly.
<code>StudentMixin(Human) / WorkerMixin(Human)</code>	Applies a mixin on top of a classically inherited class (Human), combining both inheritance styles (hybrid inheritance).
<code>new ClassName()</code>	Creates an instance (object) of the given class.
<code>console.log()</code>	Prints the result of each method call to the console for verification.

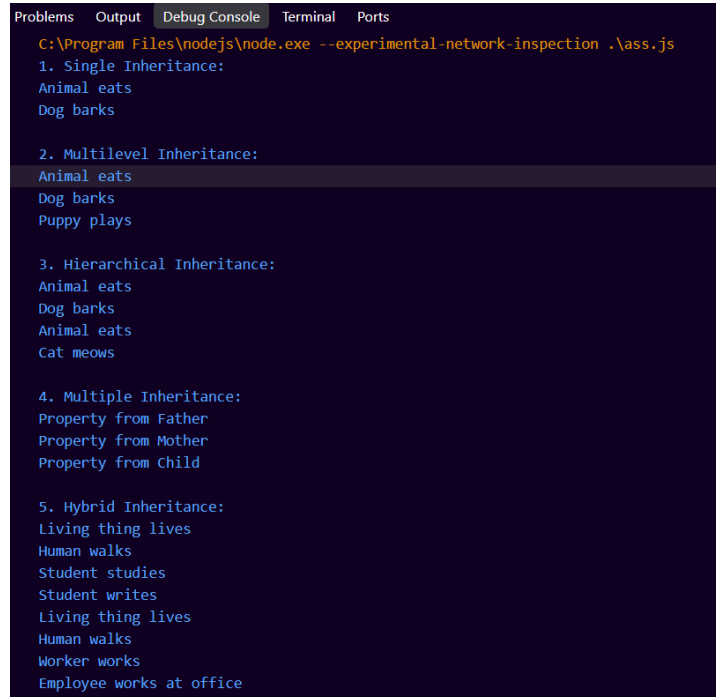
8. TEST CASES AND EXECUTION DATA

Each inheritance type was executed independently using Node.js, and the console output was recorded.

Test Case	Section / Run	Expected Result	Status
TC-01	Single Inheritance (Dog)	Prints 'Animal eats' and 'Dog barks'	Pass
TC-02	Multilevel Inheritance (Puppy)	Prints 'Animal eats', 'Dog barks' and 'Puppy plays'	Pass
TC-03	Hierarchical Inheritance (Dog3, Cat3)	Dog3 prints 'Animal eats'/'Dog barks'; Cat3 prints 'Animal eats'/'Cat meows'	Pass
TC-04	Multiple Inheritance (Child via mixins)	Prints 'Property from Father', 'Property from Mother' and 'Property from Child'	Pass
TC-05	Hybrid Inheritance (Student)	Prints 'Living thing lives', 'Human walks', 'Student studies', 'Student writes'	Pass
TC-06	Hybrid Inheritance (Employee)	Prints 'Living thing lives', 'Human walks', 'Worker works', 'Employee works at office'	Pass

9. OUTPUT

Output: node ass.js (Debug Console)



```
Problems Output Debug Console Terminal Ports
C:\Program Files\nodejs\node.exe --experimental-network-inspection .\ass.js
1. Single Inheritance:
Animal eats
Dog barks

2. Multilevel Inheritance:
Animal eats
Dog barks
Puppy plays

3. Hierarchical Inheritance:
Animal eats
Dog barks
Animal eats
Cat meows

4. Multiple Inheritance:
Property from Father
Property from Mother
Property from Child

5. Hybrid Inheritance:
Living thing lives
Human walks
Student studies
Student writes
Living thing lives
Human walks
Worker works
Employee works at office
```

Console output showing all five inheritance types executing correctly.

10. RESULT

Thus, the different types of inheritance in JavaScript — single, multilevel, hierarchical, multiple (using mixins), and hybrid inheritance — were successfully implemented using ES6 class syntax. Each type was verified by creating objects and confirming that the correct set of inherited and own methods executed as expected.